

1. How does classloader work

ClassLoaders are responsible for loading class files into the JVM. It follows the **Delegation Model**: when a class is requested, the ClassLoader first delegates the request to its parent. Only if the parent cannot find the class does the current ClassLoader attempt to load it. This ensures core Java classes are loaded only once.

2. Java access modifier scope

In Java, access modifiers define the visibility and accessibility of classes, methods, and variables. They are essential for **encapsulation**, allowing you to control how much of your code is exposed to other parts of your application.

- **public**: Accessible from anywhere.
- **protected**: Accessible within the same package and by subclasses in different packages.
- **default (no modifier)**: Accessible only within the same package.
- **private**: Accessible only within the same class.

3. What is static

static indicates that a member belongs to the class itself rather than any specific instance. Static variables are shared across all instances, and static methods can be called without creating an object. They are initialized when the class is loaded.

1. Static Variables (Class Variables)

These are shared across all instances of a class. If one object changes the value of a static variable, that change is reflected in every other object of that class.

- **Use case**: Constants (e.g., `static final double PI = 3.14159;`) or counters that track the number of objects created.

2. Static Methods (Class Methods)

These methods can be called without creating an instance of the class. They are often used for utility functions that don't depend on instance-specific data.

- **Use case**: Math libraries (e.g., `Math.sqrt()`) or factory methods.
- **Restriction**: A static method **cannot** access instance variables or instance methods directly, because it doesn't know *which* object's data to access.

3. Static Blocks

A block of code marked with **static** that executes **only once**—when the class is first loaded into memory by the ClassLoader.

- **Use case**: Initializing complex static variables that require logic (e.g., loading configuration files or native libraries).

4. Default keyword vs Java default scope

- **default keyword**: Used in interfaces to provide a default implementation for a method, and in **switch** statements to handle cases not covered by explicit labels.
- **Default scope**: An implicit access modifier (package-private) used when no visibility keyword is specified, restricting access to the current package.

5. What is OOP

Object-Oriented Programming (OOP) is a paradigm based on "objects" that contain data (fields) and code (methods). Its four pillars are:

- **Encapsulation**: Hiding internal state.
- **Inheritance**: Creating new classes from existing ones.
- **Polymorphism**: Allowing objects to be treated as instances of their parent class.
- **Abstraction**: Hiding complex implementation details.

four core principles:

1. Encapsulation

The practice of keeping the internal state of an object hidden from the outside and only exposing the necessary parts through **public** methods. This protects the integrity of the data.

- *Analogy:* You know how to operate a microwave by using the buttons (public interface), but you don't need to know the complex circuitry inside (private implementation).

2. Abstraction

Simplifying complex reality by modeling classes appropriate to the problem. It focuses on *what* an object does rather than *how* it does it.

- *Analogy:* When you drive a car, you use the steering wheel and pedals. You don't need to know how the fuel injection or engine combustion works to get from A to B.

3. Inheritance

A mechanism where one class (the child) can acquire the properties and behaviors of another class (the parent). This promotes **code reusability**.

- *Analogy:* A **Truck** class can inherit from a general **Vehicle** class. The **Truck** gets all the basic features of a **Vehicle** (like **speed** or **brand**), but adds its own specific features (like **cargoCapacity**).

4. Polymorphism

The ability for different classes to be treated as instances of the same general class through a common interface. It allows one method to perform different tasks based on the object it is acting upon.

- *Analogy:* A "Shape" class might have a method called **draw()**. A **Circle** will draw itself as a circle, and a **Square** will draw itself as a square, even though you just called **draw()** on both.

Why use OOP?

- **Modularity:** Troubleshooting is easier because each object acts as an independent unit.
- **Reusability:** Through inheritance, you can write code once and reuse it in multiple parts of your application.
- **Scalability:** It is much easier to manage large, complex software systems by breaking them down into small, manageable objects.
- **Flexibility:** Polymorphism allows systems to be easily extended with new types of objects without modifying existing code.

6. Immutable class

An immutable class is one whose state cannot be changed after creation (e.g., **String**). To create one: make the class **final**, keep all fields **private** and **final**, provide no setters, and ensure defensive copies are made for mutable field references.

To create your own immutable class, you must strictly follow these rules:

1. **Mark the class as **final**:** This prevents other classes from extending it and overriding methods to break immutability.
2. **Make all fields **private** and **final**:** **private** ensures direct access is blocked, and **final** ensures the reference/value cannot be changed after initialization.
3. **Do not provide "Setter" methods:** No method should exist that allows the modification of the object's state.
4. **Use "Defensive Copying" for mutable objects:** If your class contains a reference to a mutable object (like a **Date** or **List**), you must return a copy of that object in getters and store a copy in the constructor. Otherwise, the caller could modify the internal state through the reference.

Why use Immutability?

- **Thread Safety:** Because the state cannot change, immutable objects are inherently thread-safe. You don't need to worry about **synchronized** blocks or race conditions.

- **Safe Map Keys:** Immutable objects make perfect keys for a `HashMap` or `HashSet` because their hash code will never change after insertion.
- **Reliability:** You can pass an immutable object to other methods without worrying that they might accidentally change its state (side effects).
- **Caching:** Since they are constant, you can cache immutable objects or reuse them across the application (e.g., the Java `String Pool`).

7. How does thread pool work

A thread pool manages a pool of worker threads. When a task is submitted, it is added to a queue. If the number of running threads is less than `corePoolSize`, a new thread is created. If the queue is full, it expands up to `maximumPoolSize`. If tasks continue to arrive after the limit, the `RejectedExecutionHandler` is triggered.

A **Thread Pool** is a design pattern used to manage a collection of worker threads that perform tasks, rather than creating a new thread for every task. It promotes resource efficiency by reusing existing threads and preventing the overhead of frequent thread creation and destruction.

How It Works

When a task is submitted to a thread pool, the pool manages the execution flow using a

Task Queue and a **Worker Set**:

1. **Queueing:** When a new task is submitted, it is placed in an internal queue.
2. **Assignment:** If a thread in the pool is idle, it picks up the next task from the queue.
3. **Execution:** The thread executes the task's `run()` method.
4. **Recycling:** Once finished, the thread does not terminate; it returns to the pool to wait for the next task in the queue.

Core Components of a Thread Pool

- **Core Pool Size:** The minimum number of threads kept in the pool, even if they are idle.
- **Maximum Pool Size:** The upper limit of threads allowed to be created when the task queue is full.
- **Keep-Alive Time:** How long excess idle threads (beyond the core size) will wait before being terminated to save resources.
- **Blocking Queue:** A buffer that holds tasks waiting to be processed when all core threads are busy.

Why Use a Thread Pool?

- **Performance (Reduced Overhead):** Thread creation and destruction are expensive in terms of CPU and memory. Reusing threads significantly reduces latency.
- **Resource Management:** By limiting the maximum number of threads, you prevent your application from crashing due to an "OutOfMemoryError" or "Too many threads" errors under heavy load.
- **Responsiveness:** Tasks can be processed immediately if a thread is available, rather than waiting for a new thread to be initialized by the OS.

8. Potential problems for `newCachedThreadPool` and `newFixedThreadPool`

- **`newCachedThreadPool`:** Can create an unbounded number of threads, leading to `OutOfMemoryError: unable to create new native thread`.
- **`newFixedThreadPool`:** Uses an unbounded `LinkedBlockingQueue`. If tasks accumulate faster than they are processed, the queue consumes all available heap space, leading to `OutOfMemoryError: Java heap space`.

While `newCachedThreadPool` and `newFixedThreadPool` are powerful tools in the `java.util.concurrent` framework, using them incorrectly can lead to severe performance bottlenecks or application crashes. Here are the specific risks associated with each.

1. `newCachedThreadPool`

The "Cached" thread pool is designed to create new threads as needed and reuse them when they become available. It is intended for applications with many short-lived, asynchronous tasks.

- **Risk: Resource Exhaustion (Too Many Threads)** The `maximumPoolSize` is set to `Integer.MAX_VALUE`. If your application submits tasks faster than the threads can process them, the pool will continue to spawn new threads. This can lead to:
 - **OutOfMemoryError: unable to create new native thread:** Eventually, the OS will reject the request to create more threads.
 - **CPU Throttling:** Context switching overhead between thousands of threads will cause performance to plummet, potentially hanging the entire JVM.

2. `newFixedThreadPool`

The "Fixed" thread pool uses a constant number of threads and a shared queue for pending tasks.

- **Risk: Queue Overflow (Memory Exhaustion)** The default `FixedThreadPool` uses an **unbounded `LinkedBlockingQueue`**. If tasks arrive faster than the pool can process them, the queue will grow indefinitely.
 - **OutOfMemoryError: Java heap space:** As the queue stores millions of pending task objects, the heap memory will fill up until the JVM crashes.
- **Risk: Task Latency** Because the queue is unbounded, new tasks are accepted regardless of the system's load. If the processing rate is slower than the arrival rate, the time a task waits in the queue ("queued time") can grow to minutes, making the application appear unresponsive even if the CPU usage looks low.

Best Practices for Production

To avoid these pitfalls, avoid the `Executors` factory methods for critical production systems. Instead, instantiate the `ThreadPoolExecutor` directly so you can define **bounded** parameters:

1. **Define a Maximum Limit:** Always set a reasonable `maximumPoolSize`.
2. **Use a Bounded Queue:** Use `ArrayBlockingQueue` or a sized `LinkedBlockingQueue` instead of the default unbounded one.
3. **Define a Rejection Policy:** When the queue is full and all threads are busy, decide how to handle new tasks (e.g., `AbortPolicy` to throw an exception, or `CallerRunsPolicy` to force the submitting thread to execute the task itself, effectively slowing down the producer).

9. Lock vs synchronized

- **synchronized:** A JVM-level keyword; simple to use, automatic lock release, but lacks flexibility (cannot be interrupted, no fairness).
- **Lock:** A Java API; more powerful, supports `tryLock()`, interruptible locking, and fairness policies.

Both `synchronized` and the `Lock` interface in Java are used to achieve thread synchronization and ensure mutual exclusion. However, they provide different levels of flexibility and control.

1. `synchronized` (Intrinsic Lock)

`synchronized` is a built-in language keyword. It uses an **intrinsic lock** (or monitor lock) associated with every object.

- **Ease of Use:** Very simple to implement. You just add the keyword to a method or use it as a block.
- **Automatic:** The JVM automatically acquires the lock when entering the code and releases it when exiting (even if an exception occurs).
- **Fixed Scope:** The scope is strictly defined by the method or block structure.

- **Performance:** In modern JVMs, `synchronized` is highly optimized (biased locking, lock coarsening) and often performs similarly to `ReentrantLock`.

2. Lock (Explicit Lock)

`Lock` is an interface in `java.util.concurrent.locks` (e.g., `ReentrantLock`). It provides more advanced capabilities than intrinsic locks.

- **Flexibility:** You can acquire and release the lock in different scopes or even across different methods.
- **Non-blocking Attempts:** You can use `tryLock()`, which attempts to acquire the lock but returns `false` immediately if it is unavailable, preventing the thread from waiting indefinitely.
- **Fairness:** `ReentrantLock` allows you to set a "fairness" parameter, ensuring threads acquire the lock in the order they requested it (FIFO).
- **Interruptibility:** A thread waiting for a `Lock` can be interrupted, whereas a thread blocked waiting for a `synchronized` block cannot.

When to use which?

Use `synchronized` for most simple use cases. It is cleaner, less error-prone (no need to manually release the lock), and the JVM optimizes it very well.

Use `Lock` only when you need advanced features:

- When you need a **fair** locking policy.
- When you need to **poll** for a lock (`tryLock`).
- When you need to **interrupt** a thread that is waiting for a lock.
- When you need **multiple wait-sets** for a single lock (using `Condition` objects).

Important Note: The `finally` Trap

If you use a `Lock`, you **must** release it in a `finally` block. If you don't, and an exception occurs, the lock will never be released, potentially deadlocking your entire application.

10. wait(), notify(), notifyAll(), join(), sleep()

- **wait():** Releases the object lock and pauses the thread.
- **notify()/notifyAll():** Wakes up one or all threads waiting on that monitor.
- **join():** Pauses the current thread until the joined thread completes.
- **sleep():** Pauses for a specified time; **does not** release locks.

These methods are essential for managing thread execution, synchronization, and communication in Java. They are broadly categorized into **Object-level monitors** (for signaling) and **Thread control** (for timing and ordering).

1. The Signaling Mechanism: wait(), notify(), notifyAll()

These methods belong to the `java.lang.Object` class and must be called inside a `synchronized` block or method. They allow threads to communicate with each other regarding the state of a shared resource.

- **wait():** Causes the current thread to release the lock and enter a "waiting state" until another thread invokes `notify()` or `notifyAll()` on the same object.
- **notify():** Wakes up **one** single thread that is waiting on this object's monitor. The choice of which thread to wake is arbitrary.
- **notifyAll():** Wakes up **all** threads that are waiting on this object's monitor. This is generally safer than `notify()` to avoid potential "lost signal" deadlocks.

2. Thread Control: sleep() and join()

These methods are static or instance methods belonging to the `java.lang.Thread` class and control the timing of a thread's lifecycle.

- **sleep(long millis):** A static method that pauses the current thread for a specified amount of time.

- **Crucially:** The thread **does not release any locks** it currently holds. It simply stops execution.
- **join():** Used to force one thread to wait for the completion of another. If `threadA` calls `threadB.join()`, `threadA` will pause until `threadB` finishes its execution.

Critical Differences to Remember

1. **Lock Release:** The biggest functional difference is that `wait()` releases the monitor lock, allowing other threads to enter the `synchronized` block and potentially change the state of the object. `sleep()` and `join()` keep the lock, which can lead to deadlocks if the sleeping thread is waiting for another thread to acquire that same lock.
2. **Usage Pattern:**
 - Use `wait/notify` when you need **coordination** (e.g., a Producer-Consumer pattern where one thread waits for data to be available).
 - Use `sleep` when you need a **delay** (e.g., polling a database every few seconds).
 - Use `join` when you need **sequencing** (e.g., ensuring a "Worker" thread finishes processing data before a "ResultCollector" thread starts).

Pro-Tip: Always call `wait()` inside a `while` loop (not an `if` statement) to guard against "spurious wakeups," where a thread might wake up even if the condition it is waiting for is not actually met.

11. How to create a thread

Java

// 1. Extend Thread

```
class MyThread extends Thread { public void run() { ... } }
```

// 2. Implement Runnable

```
new Thread(() -> { ... }).start();
```

// 3. Implement Callable (with FutureTask)

```
FutureTask<Integer> task = new FutureTask<>(() -> 1);
```

```
new Thread(task).start();
```

// 4. ExecutorService (Thread Pool)

```
ExecutorService pool = Executors.newFixedThreadPool(10);
```

In Java, there are several ways to create and manage threads. While there are multiple approaches, they generally fall into two categories: the classic way (extending or implementing) and the modern, preferred way (using the `Executor` framework).

1. The Classic Ways (Foundational)

Implementing `Runnable` (Recommended)

This is the most flexible approach because it separates the task logic from the thread object. It also allows your class to extend another class, which you cannot do if you extend `Thread`.

Extending `Thread`

This is simpler but less flexible. You are forced to inherit from `Thread`, which limits your class hierarchy.

2. The Modern Way (Production-Grade)

In real-world applications, you should almost never manually create `new Thread()` objects. Instead, you should use the `ExecutorService` framework. This manages a pool of threads for you, which is significantly more efficient and easier to maintain.

Key Concepts for Success

- **run() vs start():** Never call `run()` directly. Calling `run()` just executes the method in the current thread (the main thread). You **must** call `start()`, which tells the JVM to create a new call stack and then execute your `run()` method in that new thread.

- **Lambda Expressions:** Since Java 8, if you are implementing a functional interface like `Runnable`, you can keep your code very concise using lambdas: `new Thread(() -> System.out.println("Simple thread!")).start();`
- **Thread Lifecycle:** Once a thread starts, it moves through states like *Runnable*, *Running*, *Blocked/Waiting*, and finally *Terminated*.

12. Thread lifecycle and state transfer

Threads cycle through: **NEW**, **RUNNABLE**, **BLOCKED**, **WAITING**, **TIMED_WAITING**, and **TERMINATED**. Transitions are triggered by methods like `start()`, `wait()`, `notify()`, `sleep()`, and acquiring synchronization locks.

A Java thread's lifecycle is governed by the `java.lang.Thread.State` enum.

Understanding how a thread moves between these states is vital for debugging deadlocks and performance bottlenecks.

The Thread State Machine

A thread does not simply move from *New* to *Terminated*; it flows through several intermediate states based on the operations performed.

1. NEW

The thread object has been created (using `new Thread()`), but `start()` has not yet been called. It does not exist as a separate execution thread in the JVM yet.

2. RUNNABLE

This covers both "ready to run" and "currently running" states. The thread is active, and the JVM's thread scheduler decides when it actually gets CPU time.

3. BLOCKED

The thread is alive but is waiting to acquire a **monitor lock** (e.g., waiting to enter a `synchronized` block or method held by another thread). Once the lock is released, the thread becomes **RUNNABLE** again.

4. WAITING

The thread is waiting indefinitely for another thread to perform a specific action (e.g., `object.wait()` or `thread.join()`). It will not consume CPU cycles while in this state.

5. TIMED_WAITING

Similar to **WAITING**, but with a specified timeout (e.g., `Thread.sleep(1000)`, `object.wait(1000)`, or `thread.join(1000)`). The thread will automatically transition back to **RUNNABLE** when the time expires.

6. TERMINATED

The thread has finished its execution (the `run()` method completed naturally or via an unhandled exception). It cannot be restarted.

Important Observations

- **RUNNABLE vs. RUNNING:** In Java's `Thread.State` enum, there is no separate **RUNNING** state. Whether the thread is actually executing on the CPU or waiting in the scheduler's queue, the state is reported as **RUNNABLE**.
- **The BLOCKED Trap:** If your application feels "frozen" but CPU usage is low, a thread is likely stuck in the **BLOCKED** state waiting for a monitor lock.
- **Visibility:** You can check the current state of a thread programmatically using `thread.getState()`. This is incredibly useful for logging or debugging concurrency issues in production.

Pro-Tip: If you are debugging concurrency, use the `jstack` tool. It provides a snapshot of all threads and their current states, which helps identify exactly which line of code is causing a thread to be in a **BLOCKED** or **WAITING** state.

13. runAsync / supplyAsync

- **runAsync:** Takes a `Runnable` (no return value).
- **supplyAsync:** Takes a `Supplier` (returns a result).

In Java, `runAsync` and `supplyAsync` are static factory methods introduced in **Java 8** within the `CompletableFuture` class. They are the modern, standard way to perform **asynchronous programming** without manually managing raw `Thread` objects. Both methods submit a task to the `ForkJoinPool.commonPool()` (or a custom executor) and return a `CompletableFuture` representing the eventual result.

1. `runAsync` (The "Do-er")

Use this when you want to run a task asynchronously that **does not return a result**. It accepts a `Runnable` functional interface.

- **Signature:** `static CompletableFuture<Void> runAsync(Runnable runnable)`
- **Usage:** Ideal for background logging, sending emails, or updating file systems where the calling thread just needs to "fire and forget."

2. `supplyAsync` (The "Provider")

Use this when you want to run a task asynchronously that **returns a result**. It accepts a `Supplier<T>` functional interface.

- **Signature:** `static <U> CompletableFuture<U> supplyAsync(Supplier<U> supplier)`
- **Usage:** Ideal for fetching data from a database, calling an external API, or performing heavy computations where you need the final value.

14. `allOf`, `anyOf`

- **`allOf`:** Returns a new `CompletableFuture` that completes when all provided futures complete.
- **`anyOf`:** Returns a new `CompletableFuture` that completes when any one of the provided futures completes.

In Java's `CompletableFuture` API, `allOf` and `anyOf` are powerful static factory methods used to **coordinate multiple asynchronous tasks**. They allow you to combine several independent futures into one.

1. `CompletableFuture.allOf(...)`

This method is used when you need to wait for **all** provided tasks to complete.

- **Behavior:** It returns a new `CompletableFuture<Void>`. This future completes only when every input future has finished successfully.
- **Use Case:** Gathering data from multiple microservices simultaneously, where you cannot proceed until all data sets are collected.

2. `CompletableFuture.anyOf(...)`

This method is used when you only care about the result of the **first** task to complete.

- **Behavior:** It returns a new `CompletableFuture<Object>`. It completes as soon as *any* one of the input futures completes, regardless of success or failure.
- **Use Case:** Redundancy or optimization. For example, querying the same data from three different database mirrors and taking the result from whichever responds the fastest.

15. List vs Set

- **List:** Ordered collection, allows duplicates, provides index-based access (e.g., `ArrayList`).
- **Set:** Unordered (usually), does not allow duplicates (e.g., `HashSet`).

In Java, `List` and `Set` are both sub-interfaces of the `Collection` interface, but they are designed for completely different data management requirements. The fundamental difference lies in **ordering** and **uniqueness**.

1. The Core Differences

- **List (Ordered Collection):**

- **Duplicates:** Allows duplicate elements.
- **Ordering:** Maintains the **insertion order**. Elements can be accessed by their index (position).
- **Common Implementations:** `ArrayList`, `LinkedList`, `Vector`.
- **Analogy:** A grocery list—if you write "milk" twice, it appears twice, and the order you write them in matters.
- **Set (Unordered/Unique Collection):**
 - **Duplicates:** Does **not** allow duplicates. If you try to add a duplicate, the operation will fail or overwrite the existing element.
 - **Ordering:** Does not guarantee order (with the exception of specific implementations like `LinkedHashSet` or `TreeSet`).
 - **Common Implementations:** `HashSet`, `LinkedHashSet`, `TreeSet`.
 - **Analogy:** A guest list for an event—each name appears only once, and the order is usually not the primary concern.

When to Use Which?

Use a `List` when:

- You need to maintain the order in which items were added.
- You need to access elements frequently by their index (e.g., `list.get(5)`).
- Your data naturally allows for duplicates.

Use a `Set` when:

- **Uniqueness is a priority:** You must ensure no two elements are the same (e.g., unique user IDs, tags on a blog post).
- **Performance of search:** You need to check if an item exists (`contains()`) frequently; `HashSet` provides $O(1)$ time complexity for this, which is significantly faster than a `List`.

Important Implementation Variations

- **`HashSet`:** Offers the best performance for adding and searching but does not guarantee any order.
- **`LinkedHashSet`:** Maintains the **insertion order** of elements while still ensuring uniqueness.
- **`TreeSet`:** Keeps elements in **sorted order** (e.g., natural ordering or custom `Comparator`), though it is slightly slower due to the overhead of maintaining the tree structure.

Pro-Tip: If you have a `List` full of duplicates and want to remove them while maintaining performance, a common pattern is to pass the `List` into the constructor of a `HashSet`, then back into a `List`: `List<String> uniqueList = new ArrayList<>(new HashSet<>(originalList));`

16. `LinkedList` vs `ArrayList`

- **`ArrayList`:** Backed by an array. Fast random access ($O(1)$), slow insertion/deletion ($O(n)$).
- **`LinkedList`:** Backed by a doubly-linked list. Fast insertion/deletion ($O(1)$), slow random access ($O(n)$).

Choosing between `ArrayList` and `LinkedList` depends on how you plan to use the data structure, as they offer different performance characteristics based on their internal implementations.

Internal Architecture

- **`ArrayList`:** Backed by a **dynamically resizing array**. Elements are stored in contiguous memory locations.
- **`LinkedList`:** A **doubly-linked list**. Each element (node) stores a reference to the previous node and the next node, scattered across memory.

Key Trade-offs

1. Why choose **ArrayList**?

- **Superior for Reads:** If your application performs many "read" operations (e.g., retrieving items by index), **ArrayList** is significantly faster because array access is direct.
- **Cache Locality:** Because elements are in contiguous memory, **ArrayList** is much more "CPU cache-friendly," leading to better performance in tight loops.
- **General Purpose:** It is the default, go-to list implementation for most Java applications.

2. Why choose **LinkedList**?

- **Frequent Additions/Removals:** If your application involves frequent insertion or removal of elements in the middle of the list (and you already have an **Iterator** at that position), **LinkedList** is more efficient because it only requires updating a few pointers rather than shifting an entire array of elements.
- **Queue/Deque Functionality:** **LinkedList** implements the **Deque** interface, making it an excellent choice for implementing Queues (FIFO) or Stacks (LIFO).

When to Use Which?

Rule of Thumb: Use **ArrayList** by default. It is almost always more efficient due to better cache locality and lower memory overhead. Only switch to **LinkedList** if you have profiled your application and determined that frequent insertions/deletions in the middle of a large list are a specific bottleneck.

A Note on Performance

While **LinkedList** technically has "constant time" ($O(1)$) insertion, you must **already be at the position** to get that performance. If you have to traverse the list (e.g., `list.get(i)`) to find the insertion point, the total operation becomes $O(n)$ anyway, which often negates the advantage over **ArrayList**.

17. Collections used for what

The **Collections** class is a utility class containing static methods to operate on or return collections, such as `sort()`, `reverse()`, `synchronizedList()`, and `emptyList()`.

Detailed Usage Scenarios

1. Lists (**List** Interface)

- **Best for:** Maintaining insertion order and accessing elements by position.
- **ArrayList:** Use this as your default list. It is fast for reading and random access.
- **LinkedList:** Use this if you are building a queue or need to add/remove from the front of the list frequently.

2. Sets (**Set** Interface)

- **Best for:** Eliminating duplicates and checking for the presence of an item.
- **HashSet:** Extremely fast ($O(1)$) for adding, removing, and checking existence. Use this when order doesn't matter.
- **LinkedHashSet:** Use this when you need the speed of a **HashSet** but also need to maintain the order in which items were inserted.
- **TreeSet:** Use this when you need elements to be sorted according to a natural order or a custom **Comparator**.

3. Maps (**Map** Interface)

- **Best for:** Key-value pairs (dictionary style).
- **HashMap:** The standard choice. Offers $O(1)$ performance for looking up a value by its key.
- **TreeMap:** Use this if you need to iterate through the map based on the sorted order of keys.
- **LinkedHashMap:** Use this if you need to maintain the order in which entries were inserted into the map.

Key Considerations for Your Choice

- **Concurrency:** Most standard collections are **not thread-safe**. If you are working in a multi-threaded environment, use the `java.util.concurrent` versions (e.g., `ConcurrentHashMap`, `CopyOnWriteArrayList`).
- **Memory Efficiency:** `ArrayList` and `HashMap` are generally more memory-efficient than their "linked" counterparts (like `LinkedList`) because they avoid the overhead of storing extra pointer references for every node.
- **Performance:** If you find yourself doing `list.contains()` repeatedly on a large dataset, you are likely using the wrong data structure. Switch to a `Set` for that specific task to move from $O(n)$ to $O(1)$ complexity.

Pro-Tip: Always code to the interface, not the implementation.

- **Correct:** `List<String> list = new ArrayList<>();`
- **Incorrect:** `ArrayList<String> list = new ArrayList<>();`
This allows you to swap the implementation later (e.g., switching `ArrayList` to `LinkedList`) without breaking your code.

18. What is hash collision

A hash collision occurs when two distinct keys produce the same hash code, requiring the map to resolve the conflict (e.g., via linked lists or balanced trees in a `HashMap` bucket).

A **hash collision** occurs when two distinct input keys produce the exact same hash value (index) when processed by a hash function.

In an ideal world, a hash function would be perfect, meaning every unique input results in a unique hash. However, because hash functions map an infinite number of possible inputs to a finite number of output buckets (e.g., indices in an array), collisions are mathematically inevitable according to the **Pigeonhole Principle**.

How Hash Tables Handle Collisions

Since a collision cannot be completely eliminated, hash tables (like `HashMap` in Java) use specific strategies to manage them so that both items can still be stored.

1. Chaining (The Java Approach)

Instead of storing a single object at an array index, each "bucket" stores a **linked list** (or a balanced tree) of objects. If a collision occurs, the new entry is simply added to the list at that specific index.

- **When searching:** The hash table finds the correct bucket and then performs a linear search (or tree traversal) through the list to find the matching key.

2. Open Addressing

When a collision occurs, the hash table looks for the next empty "slot" in the array using a specific probe sequence (e.g., checking the next index, the one after that, etc.).

- **Pros:** Better cache locality as all data is stored within the array.
- **Cons:** The table can fill up quickly, and deleting items becomes more complex.

Why Collisions Matter for Performance

While modern hash tables handle collisions gracefully, frequent collisions are a major performance issue:

- **Best Case:** When there are no collisions, retrieving an item from a hash table is **$O(1)$** (constant time).
- **Worst Case:** If a hash function is poorly designed and causes many collisions, the table effectively becomes a linked list. Retrieving an item then degrades to **$O(n)$** , making it as slow as searching a standard list.

How Java Optimizes This

Java's `HashMap` has built-in mechanisms to minimize the impact of collisions:

1. **Spread Hash:** It applies a secondary hash function to the object's `hashCode()` to ensure bits are distributed more uniformly across the buckets.
2. **Treeification (Since Java 8):** If a single bucket becomes too crowded (the number of items in a linked list exceeds a threshold, usually 8), Java automatically converts

that linked list into a **balanced red-black tree**. This improves the worst-case search performance from **$O(n)$ to $O(\log n)$** .

Key Takeaway: Always override both `equals()` and `hashCode()` correctly in your custom objects. If `hashCode()` is poorly implemented (e.g., it always returns the same number), you will force all objects into a single bucket, completely destroying the performance benefits of using a `HashMap`.

19. Checked vs unchecked exceptions

- **Checked:** Must be handled via `try-catch` or declared with `throws` (e.g., `IOException`).
- **Unchecked:** Runtime exceptions that do not require explicit handling (e.g., `NullPointerException`).

In Java, exceptions are divided into two main categories based on whether they are checked by the compiler at compile-time. This distinction is a fundamental part of Java's robust error-handling design.

1. Checked Exceptions

These are exceptions that the compiler **forces** you to handle. They represent conditions that a well-written application should anticipate and recover from (e.g., file not found, network timeout).

- **Handling:** You **must** either catch them with a `try-catch` block or declare them in the method signature using the `throws` keyword.
- **Inheritance:** All classes that inherit from `java.lang.Exception` (but **not** `RuntimeException`) are checked exceptions.
- **Examples:** `IOException`, `SQLException`, `FileNotFoundException`.

2. Unchecked Exceptions

These are exceptions that the compiler **does not force** you to handle. They usually represent programming errors or unexpected runtime conditions that you often cannot "recover" from programmatically.

- **Handling:** You are not required to catch or declare them, though you certainly can.
- **Inheritance:** All classes that inherit from `java.lang.RuntimeException` are unchecked exceptions.
- **Examples:** `NullPointerException`, `ArrayIndexOutOfBoundsException`, `IllegalArgumentException`.

Key Design Philosophies

- **Use Checked Exceptions** when you want to force the caller of your method to explicitly acknowledge and deal with a potential failure. It makes the "contract" of the method more explicit.
- **Use Unchecked Exceptions** for logic errors that are truly "exceptional" and often stem from misuse of an API (like passing a null value where an object is expected).

A Note on Best Practices: While checked exceptions were intended to increase reliability, many modern Java developers prefer to wrap them in unchecked exceptions (or custom runtime exceptions) to avoid "exception pollution" in method signatures. This keeps the code cleaner while still allowing the application to fail gracefully at the top level.

20. finally, final, finalize

- **finally:** A block guaranteed to execute after a `try-catch`.
- **final:** A modifier for classes (cannot inherit), methods (cannot override), or variables (constant).
- **finalize:** A deprecated method called by the GC before object reclamation.

Although these three terms look similar, they serve completely different purposes in Java. Here is the breakdown:

1. **final** (The Keyword)

final is a modifier used to indicate that an entity cannot be changed or modified once it is defined.

- **Final Variable:** The value cannot be reassigned (it becomes a constant).
- **Final Method:** The method cannot be overridden by subclasses.
- **Final Class:** The class cannot be extended (inherited).

2. **finally** (The Block)

finally is a block used in **try-catch** structures. It ensures that a piece of code **always executes**, regardless of whether an exception was thrown or caught.

- **Purpose:** It is typically used for cleanup code, such as closing file streams, database connections, or releasing locks.
- **Behavior:** Even if you have a **return** statement in the **try** or **catch** block, the **finally** block will still execute before the method exits.

3. **finalize()** (The Method)

finalize() is a method inherited from **java.lang.Object**. The garbage collector calls this method before reclaiming an object's memory.

- **Purpose:** It was originally intended for cleanup, but it is **highly discouraged**.
- **Status:** It has been **deprecated since Java 9**. You should never rely on it, as there is no guarantee *when* (or if) the garbage collector will call it.
- **Alternative:** Use **AutoCloseable** and the **try-with-resources** statement for reliable resource management.

Crucial Warning: Never use **finalize()** for critical resource cleanup. Because its timing is non-deterministic, your application could run out of file descriptors or memory long before the garbage collector decides to run **finalize()**. Always use **try-with-resources** for modern Java development.

21. Why clean up I/O resources in finally

I/O resources are finite. If an exception occurs, code following the open resource call may be skipped. Placing cleanup in **finally** ensures the resource is closed regardless of whether an exception is thrown, preventing memory/file handle leaks.

Cleaning up I/O resources (like files, network sockets, or database connections) in a **finally** block—or more accurately, using **try-with-resources**—is critical to maintaining the health and stability of your application.

Here is why this practice is non-negotiable:

1. Preventing Resource Leaks

Operating systems have a strict limit on the number of "file descriptors" or "sockets" a single process can open simultaneously.

- **The Problem:** If you open a file but fail to close it, that descriptor remains "in use" by your application even after you are done with it.
- **The Consequence:** If your code is inside a loop or runs frequently, you will eventually hit the system's limit. Once exhausted, your application will crash with an **IOException: Too many open files** and will be unable to open any new resources until it is restarted.

2. Ensuring Cleanup During Exceptions

This is the primary reason for using **finally** or **try-with-resources**.

- **The Scenario:** If an error occurs (like an **IOException**) while reading or writing to a file, the code execution immediately jumps to the **catch** block.
- **The Risk:** If your **.close()** method is at the bottom of your **try** block, **it will be skipped entirely** when an exception is thrown.
- **The Fix:** Putting the close operation in a **finally** block guarantees it runs whether the operation succeeded or failed.

The Modern Standard: **try-with-resources**

While **finally** blocks work, they are verbose and error-prone (you have to handle null checks and potential exceptions within the **finally** block itself).

Since Java 7, the **try-with-resources** statement is the industry-standard way to handle I/O. It automatically closes any resource that implements the **AutoCloseable** interface as soon as the block exits.

Summary of Benefits

- **Deterministic Cleanup:** Resources are closed immediately when the **try** block finishes, rather than waiting for the Garbage Collector.
- **Suppressed Exceptions:** If an exception occurs in the **try** block AND an exception occurs while closing the resource, **try-with-resources** correctly associates the primary exception with the developer, while "suppressing" the closing exception, preventing you from losing the root cause of your bug.
- **Cleaner Code:** It significantly reduces "boilerplate" code, making your methods easier to read and maintain.

22. Runtime Exception

A subclass of **RuntimeException** that occurs at runtime. Example: `int result = 10 / 0;` triggers an **ArithmeticException**.

A **Runtime Exception** is an exception that occurs during the execution of a program, representing a condition that the application usually cannot recover from. In Java, these are defined by the **RuntimeException** class (and its subclasses), which inherits from **Exception**.

These are classified as **Unchecked Exceptions**, meaning the Java compiler does not force you to handle them with a **try-catch** block or declare them in a method signature using **throws**.

Why "Unchecked"?

The design philosophy behind Runtime Exceptions is that they generally represent **programming errors** or **logic bugs**. Unlike checked exceptions (which usually signal external issues like a missing file), runtime exceptions occur because the code did something invalid.

- **Logic Errors:** Dividing by zero or accessing an invalid index.
- **API Misuse:** Passing `null` to a method that doesn't accept it, or passing an invalid argument value.
- **State Issues:** Attempting to perform an operation that is invalid given the current state of an object.

23. What is Map Interface

A collection that maps keys to values, with no duplicate keys allowed. Each key maps to at most one value.

The **Map interface** is a fundamental part of the Java Collections Framework that represents a collection of key-value pairs. Unlike **List** or **Set**, a **Map** is **not** a true **Collection** in the hierarchical sense (it does not extend the **Collection** interface), but it is treated as one for practical purposes.

A **Map** maps unique **keys** to specific **values**. Each key can map to at most one value.

Core Characteristics

- **Unique Keys:** Every key in a map must be unique. If you try to add a duplicate key, the new value will overwrite the old one.
- **Non-unique Values:** Multiple keys can map to the same value.
- **Lookup Efficiency:** The primary power of a map is its ability to find a value instantly using its associated key.

Important Considerations

1. **Keys must be immutable:** If you use a mutable object as a key and then change that object, the **Map** will likely lose track of the entry because the **hashCode** will change. Always use **String**, **Integer**, or custom immutable objects as keys.

2. **null handling:** `HashMap` allows one `null` key and multiple `null` values. `TreeMap` does **not** allow `null` keys (because it cannot compare them for sorting).
3. **Thread Safety:** Standard `Map` implementations (like `HashMap`) are **not** thread-safe. If multiple threads access the same map and at least one thread modifies it structurally, you must use `ConcurrentHashMap` or wrap it with `Collections.synchronizedMap()`.

Pro-Tip: When iterating over a map, do not iterate over the keys and then call `get()` inside the loop (this is inefficient). Instead, iterate over the `entrySet()`. This allows you to access both the key and the value in a single pass.

24. How does HashMap work

It uses a hash table. It calculates an index from the key's hash code. It stores values in "buckets." In case of collision, it uses a linked list or, in modern Java, a red-black tree (if the bucket size exceeds 8).

A `HashMap` in Java is a high-performance data structure that provides constant-time performance ($O(1)$) for basic operations like `put()` and `get()` by using a technique called **hashing**.

It works by storing entries in an internal array of "buckets."

The Internal Mechanism

1. **Hashing:** When you put a key-value pair into the map, the `HashMap` calls the `hashCode()` method on your key. It then applies a secondary hash function to that integer to ensure the distribution is uniform across the buckets.
2. **Indexing:** The resulting hash is converted into an index using the formula `index = hash % array.length`. This determines which bucket the entry belongs to.
3. **Storage:** The entry is stored in the bucket. If the bucket is empty, the entry is placed there. If the bucket is already occupied, a **collision** occurs.

Handling Collisions

Since different keys can result in the same hash index (a collision), `HashMap` uses two main strategies to ensure data integrity:

- **Chaining:** Each bucket actually holds a linked list (or a balanced tree) of entries. If two keys map to the same index, they are both stored in that bucket's list.
- **Treeification (Java 8+):** To prevent performance from degrading to $O(n)$ in high-collision scenarios, Java monitors the size of these lists. Once a list exceeds a threshold (8 entries), the `HashMap` automatically converts that linked list into a **balanced Red-Black tree**, reducing the lookup time to $O(\log n)$.

The `get()` Operation

When you call `map.get(key)`:

1. The map calculates the hash of the key again.
2. It identifies the correct bucket using the hash.
3. It iterates through the linked list (or searches the tree) in that bucket.
4. It uses the `equals()` method to find the entry where the key matches exactly.

Key Performance Tuning

- **Initial Capacity:** The default size of the array is 16. If you know you will store thousands of items, set a higher initial capacity to prevent the map from constantly "rehashing" (which is an expensive process of creating a new, larger array and moving all existing entries over).
- **Load Factor:** The default is 0.75. This means the `HashMap` will resize (double its size) when it is 75% full.

Critical Requirement: For a `HashMap` to work, the keys **must** correctly implement `hashCode()` and `equals()`. If two objects are equal according to `equals()`, they **must** return the same `hashCode()`. If they do not, the `HashMap` will treat them as different keys, even if they are logically the same, leading to data duplication and retrieval failures.

25. Overriding hashCode and equals

These must be overridden together to maintain the contract: if `a.equals(b)` is true, then `a.hashCode()` must be equal to `b.hashCode()`. This is essential for `HashMap` and `HashSet` to locate objects correctly.

When you store custom objects as keys in a `HashMap` or elements in a `HashSet`, you **must** override both `equals()` and `hashCode()`. If you don't, your objects will not behave correctly in these collections, leading to data loss or "missing" items.

The Fundamental Contract

Java dictates a strict contract between these two methods:

1. **Consistency:** If two objects are equal according to the `equals()` method, they **must** return the same integer result from `hashCode()`.
2. **Unidirectionality:** If two objects have the same `hashCode()`, they do **not** necessarily have to be equal. However, if they have different hash codes, they **must not** be equal.
3. **Stability:** The `hashCode()` value must remain consistent during the lifetime of the object, provided that the information used in the `equals()` comparison does not change.

Why the Contract Matters

When you call `map.put(key, value)`, the `HashMap` uses the `hashCode()` to decide **which bucket** to place the object in.

- **If you override `equals()` but NOT `hashCode()`:** Two logically equal objects will have different hash codes. The `HashMap` will place them in different buckets, meaning `map.get()` will never find the duplicate key, effectively breaking the map's ability to identify duplicates.
- **If you override `hashCode()` but NOT `equals()`:** The map might place two equal objects in the same bucket, but it will use the default `equals()` (which compares memory addresses), failing to recognize that they are logically the same object.

26. String vs StringBuilder vs StringBuffer

- **String:** Immutable.
- **StringBuilder:** Mutable, not thread-safe, fast.
- **StringBuffer:** Mutable, thread-safe, slower.

In Java, `String`, `StringBuilder`, and `StringBuffer` are all used to store and manipulate sequences of characters, but they differ significantly in their mutability and thread safety.

1. String (Immutable)

A `String` in Java is **immutable**. Once created, its value cannot be changed. Any operation that appears to modify a string (like `concat` or `substring`) actually creates a new `String` object in memory.

- **Memory Impact:** Because of immutability, frequent concatenations in a loop create many temporary, discarded objects, which puts heavy pressure on the Garbage Collector.
- **Thread Safety:** Inherently thread-safe because it cannot be modified.

2. StringBuffer (Mutable & Thread-Safe)

`StringBuffer` is **mutable**, meaning you can change the content of the same object without creating new ones.

- **Thread Safety:** Every method in `StringBuffer` is **synchronized**. This makes it safe for use by multiple threads simultaneously.
- **Performance:** Because of the overhead of synchronization (locking/unlocking), it is **slower** than `StringBuilder`.

3. StringBuilder (Mutable & Not Thread-Safe)

`StringBuilder` is also **mutable**, but it is **not synchronized**.

- **Performance:** Since it does not incur the performance penalty of synchronization, it is the **fastest** of the three for string manipulation.
- **Thread Safety:** Not thread-safe. Use this only when string manipulation happens within a single thread.

How to Choose

1. **Use `String`** for constant values, method parameters, and anywhere immutability is preferred for clarity or thread safety.
2. **Use `StringBuilder`** for almost all string manipulation tasks, especially inside loops (e.g., building a large SQL query or a CSV string). It is the default, modern choice.
3. **Use `StringBuffer`** only if you are working in a highly multi-threaded environment where you absolutely need to share a single mutable string buffer across threads.

Pro-Tip: The compiler often optimizes simple string concatenations (like `"a" + "b" + "c"`) using `StringBuilder` behind the scenes. However, you should still explicitly use `StringBuilder` if you are performing concatenations inside a loop, as the compiler's optimization may not cover that scenario effectively.

27. HashMap vs ConcurrentHashMap

- **HashMap:** Not thread-safe, allows nulls.
- **Hashtable:** Thread-safe, but uses full-table locking (slow).
- **ConcurrentHashMap:** Thread-safe, highly concurrent, uses granular locking (CAS + Synchronized).

These three classes all implement the `Map` interface and store key-value pairs, but they differ fundamentally in how they handle thread safety, performance, and `null` values.

1. Hashtable (The Legacy)

`Hashtable` is one of the oldest collection classes in Java. It is largely considered **obsolete**.

- **Thread Safety:** Every method is **synchronized**.
- **Performance:** Very slow because all threads must compete for a single "global" lock on the entire object to perform any operation.
- **null values:** It does **not** allow `null` keys or `null` values.
- **Verdict:** Avoid using it in modern development.

2. HashMap (The Standard)

`HashMap` is the standard, high-performance map for non-threaded environments.

- **Thread Safety:** **Not thread-safe**. If multiple threads modify it concurrently, it can lead to data corruption or infinite loops.
- **Performance:** Extremely fast; optimized for single-threaded access.
- **null values:** Allows one `null` key and multiple `null` values.
- **Verdict:** Use this by default for all standard, single-threaded applications.

3. ConcurrentHashMap (The Modern Multi-threaded Choice)

`ConcurrentHashMap` was designed specifically for high-concurrency environments.

- **Thread Safety:** Thread-safe, but designed differently than `Hashtable`. Instead of locking the entire map, it uses **"lock striping"** or bucket-level locking. This allows multiple threads to read and write to different parts of the map simultaneously without blocking each other.
- **Performance:** Much higher throughput than `Hashtable` under heavy load.
- **null values:** Does **not** allow `null` keys or `null` values.
- **Verdict:** The go-to choice for multi-threaded, high-performance applications.

Key Technical Differences

- **Locking Strategy:**
 - `Hashtable` uses a single lock for the entire map.

- `ConcurrentHashMap` divides the map into segments (or buckets) and only locks the portion being modified, allowing significantly more parallelism.
- **Iterator Behavior:**
 - `HashMap` iterators are **fail-fast** (they throw `ConcurrentModificationException` if the map is modified while iterating).
 - `ConcurrentHashMap` iterators are **weakly consistent** (they are designed to be used safely while the map is being updated, without throwing exceptions).

Pro-Tip: If you absolutely need a synchronized map but don't want to use `ConcurrentHashMap`, you can use `Collections.synchronizedMap(new HashMap<>())`. However, `ConcurrentHashMap` is almost always the superior choice because it offers better concurrency and performance.

28. Comparator vs Comparable

- **Comparable:** Implemented by the class itself to define "natural" ordering (uses `compareTo`).
- **Comparator:** A separate class implemented to define custom ordering (uses `compare`).

In Java, both `Comparable` and `Comparator` are interfaces used to define the sorting logic for objects. While they achieve similar results, they differ in *where* and *how* that logic is defined.

1. Comparable (The "Natural" Order)

The `Comparable` interface is implemented by the class itself. It defines the **natural ordering** of an object—the way that objects of that class should be sorted by default.

- **Interface Method:** `int compareTo(T other)`
- **Where it lives:** Inside the class whose objects you want to sort.
- **Best for:** Defining the "obvious" way to sort, like sorting `String` alphabetically or `Integer` numerically.

2. Comparator (The "Custom" Order)

The `Comparator` interface is implemented by a separate, external class. It allows you to define **multiple sorting strategies** for the same object, without modifying the object's class definition.

- **Interface Method:** `int compare(T o1, T o2)`
- **Where it lives:** In an external class or a lambda expression.
- **Best for:** When you need to sort by different criteria (e.g., sort `Student` by `name`, then by `id`, then by `gpa`).

How to decide which to use?

- **Choose `Comparable`** if you are defining the single, primary way your class should be sorted (e.g., `Date` is always sorted by time, `String` is always sorted alphabetically).
- **Choose `Comparator`** if you need flexible, dynamic sorting (e.g., a GUI table where the user can click different column headers to sort by "Price," "Date," or "Category").

Pro-Tip: You can combine these! You can use the natural order defined by `Comparable` as your default, and use `Comparator` for specialized sorting scenarios. If you use `Collections.sort(list)` or `list.sort(null)`, it will automatically use the `Comparable` implementation.

29. Overriding vs overloading

- **Overloading:** Multiple methods in the same class with the same name but different parameters.

- **Overriding:** Providing a specific implementation for a method already defined in a parent class.

Overloading and **Overriding** are two fundamental concepts in Java that allow for polymorphism, but they operate at different stages of the development process and serve different purposes.

1. Method Overloading (Compile-Time Polymorphism)

Overloading occurs when you have **multiple methods in the same class** with the same name, but with **different parameter lists** (different number of arguments, different types, or both).

- **Purpose:** To provide multiple ways to perform a similar action (e.g., `print()` an integer, `print()` a string, `print()` an object).
- **Binding:** The compiler determines which method to call at compile-time (Static Binding).
- **Requirements:**
 - Same method name.
 - Different parameter signatures.
 - Can have different return types (as long as parameters are different).
 - Can have different access modifiers.

2. Method Overriding (Runtime Polymorphism)

Overriding occurs when a **subclass** provides a specific implementation for a method that is already defined in its **superclass**.

- **Purpose:** To provide a specific behavior for a method already defined in the parent class (e.g., a `Shape` class has a `draw()` method, but `Circle` and `Square` implement it differently).
- **Binding:** The JVM determines which method to call at runtime based on the actual object type (Dynamic Binding).
- **Requirements:**
 - Same method name and exact same parameter signature.
 - Must be an inheritance relationship (parent-child).
 - Cannot have a more restrictive access modifier than the parent.
 - Return type must be the same or a covariant type (a subclass of the original return type).

Pro-Tip: Always use the `@Override` annotation when you intend to override a method. It tells the compiler to check your work; if you accidentally change the signature (e.g., adding an extra parameter), the compiler will throw an error, alerting you that you are not actually overriding the parent method as you intended.

30. Primitive type vs reference type

- **Primitive:** Store actual values (e.g., `int`, `boolean`, `double`).
- **Reference:** Store memory addresses pointing to objects (e.g., `String`, `Integer`, user-defined objects).

In Java, the distinction between **Primitive** and **Reference** types is fundamental to how data is stored, managed in memory, and passed between methods.

1. Primitive Types

Primitive types are the most basic data types in Java. They are **not objects**. They store the actual value directly in the memory location allocated for the variable.

- **Storage:** Stored directly on the **Stack**.
- **Performance:** Extremely fast and memory-efficient.
- **Default Values:** Every primitive type has a predefined default value (e.g., `int` defaults to `0`, `boolean` to `false`).
- **The 8 Primitives:** `byte`, `short`, `int`, `long`, `float`, `double`, `boolean`, `char`.

2. Reference Types

Reference types point to **objects**. The variable does not hold the data itself; instead, it holds a **memory address (a reference)** that points to where the actual object is stored in the **Heap**.

- **Storage:** The variable (the reference) is on the Stack, but the object itself lives on the **Heap**.
- **Performance:** Slightly slower than primitives due to the overhead of object creation and memory access (dereferencing).
- **Default Value:** `null` (meaning it points to nothing).
- **Examples:** All Classes, Interfaces, Arrays, and Enums (e.g., `String`, `ArrayList`, `Integer`, `Scanner`).

Why this matters (Key Behavioral Differences)

1. Passing to Methods

- **Primitive types** are passed **by value**. A copy of the value is made. Modifying the variable inside a method does **not** change the original variable outside the method.
- **Reference types** are also passed "by value" of the reference. You are passing a copy of the memory address. This means if you modify the **object's state** (e.g., `list.add()`), the change is reflected globally. However, if you point the reference to a *new* object, it won't affect the original reference.

2. Wrapper Classes

Java provides "Wrapper Classes" (e.g., `Integer` for `int`, `Double` for `double`) to allow primitives to be treated as objects when needed—such as when working with Java Collections (which cannot store primitives directly). Java handles the conversion between these automatically via **Autoboxing** and **Unboxing**.

Pro-Tip: When comparing values, always use `==` for primitives. For reference types, `==` only checks if they point to the **same memory address**. To check if the *content* of two objects is the same, you must use the `.equals()` method.

31. How does JVM work

The JVM loads bytecode (`.class`), verifies it, and executes it via the Interpreter or JIT (Just-In-Time) compiler, managing memory and resources automatically.

The Java Virtual Machine (JVM) is the engine that allows Java to be "write once, run anywhere." It acts as an abstraction layer between your compiled Java code and the underlying hardware.

The Execution Process

The journey from your code to machine execution involves three main stages:

1. Compilation

When you write Java code (`.java` file), the **Java Compiler (javac)** converts it into **Bytecode** (`.class` file). Bytecode is a platform-independent, intermediate representation of your code.

2. Class Loading

The **Class Loader** subsystem brings your bytecode into the JVM memory. It dynamically loads, links, and initializes the necessary classes just-in-time when they are first needed.

3. Execution (The Heart of the JVM)

The JVM executes the bytecode using two main components:

- **Interpreter:** Reads the bytecode line-by-line and executes it immediately. This is fast to start but slow to run for long periods.
- **Just-In-Time (JIT) Compiler:** Monitors code as it runs. When it identifies "hot spots" (frequently executed code), the JIT compiles that specific bytecode into native machine code (the language the CPU speaks). Subsequent calls run at full hardware speed.

JVM Memory Architecture

The JVM manages memory automatically through several distinct areas:

- **Heap:** Where all **objects** are stored. This is the largest part of memory and is managed by the Garbage Collector.
- **Stack:** Each thread has its own stack. It stores **local variables** and method call frames. When a method returns, its stack frame is destroyed.
- **Method Area:** Stores class-level data, constants, and static variables.

Garbage Collection (GC)

Unlike languages like C++, you don't manually free memory. The JVM runs an automatic **Garbage Collector** that periodically identifies objects that are no longer reachable from any part of your code (i.e., they are no longer in use) and reclaims their memory.

Why the JVM is so Powerful

1. **Platform Independence:** Because the JVM handles the communication with the OS (Windows, Linux, macOS), you don't need to recompile your code for different hardware.
2. **Safety:** The JVM performs "Bytecode Verification" to ensure that the code is not doing anything malicious or memory-unsafe before it is ever executed.
3. **Optimization:** The JIT compiler is incredibly sophisticated. It continuously optimizes your code while it runs, often making Java code run as fast as natively compiled C++ code in long-running applications.

Key takeaway: The JVM is not just an interpreter; it is a sophisticated **adaptive compiler** that learns how your code behaves while it's running and optimizes it in real-time to match the capabilities of the CPU it's currently on.

32. JVM memory data model

The JVM divides memory into the **Heap** (objects), **Stack** (local variables/frames), **Method Area** (classes/metadata), and **Program Counter Register**.

The JVM Memory Data Model is the blueprint for how Java manages resources during execution. It is split into two primary categories: areas that are **shared** among all threads and areas that are **private** to each individual thread.

1. Thread-Private Areas (Created per Thread)

These areas are unique to each executing thread and are automatically cleaned up when the thread terminates.

- **JVM Stack:** Stores local variables, method parameters, and partial results. Each method call creates a new **Stack Frame** within this stack.
- **Native Method Stack:** Similar to the JVM stack, but used for native methods (code written in languages like C or C++).
- **Program Counter (PC) Register:** Holds the address of the current instruction being executed by the thread.

2. Shared Areas (Shared by all Threads)

These areas contain data accessible to every part of your application.

- **Heap:** The largest memory area. This is where **all objects and arrays** created with the **new** keyword reside. It is the primary target for the **Garbage Collector**.
- **Method Area:** Stores class structures (the metadata), constant pools, static fields, and the code for methods.
 - *Note:* Since Java 8, this is managed in **Metaspace** (which uses native memory rather than the heap).

The "Stack vs. Heap" Interaction

Understanding how these two interact is essential for debugging issues like

StackOverflowError and **OutOfMemoryError**:

1. **Allocation:** When you do `User u = new User();`, the reference variable **u** is stored on the **Stack**, while the actual **User** object is created on the **Heap**.
2. **Access:** The stack variable **u** contains a pointer (memory address) to the location in the heap where the object lives.
3. **Cleanup:**

- **Stack:** Once a method completes, its stack frame is "popped," and its local variables are immediately reclaimed.
- **Heap:** Objects remain until they are no longer reachable (no references left). The Garbage Collector then periodically scans the heap to delete these "unreachable" objects.

Common Memory Errors

- **StackOverflowError:** Occurs when a thread reaches its stack limit, usually due to **infinite recursion** (calling the same method repeatedly without a base case).
- **OutOfMemoryError:** Occurs when the JVM has run out of space in the **Heap** (or Metaspace) and the Garbage Collector cannot reclaim enough memory to allocate new objects. This often happens if you have a "memory leak" (unintentionally holding references to objects that are no longer needed).

Pro-Tip: If you are monitoring memory, keep in mind that the **Stack** size is usually quite small (often 1MB per thread), while the **Heap** is large and is where most of your application's data lives. Most performance tuning is focused on optimizing the Heap and the Garbage Collector.

33. How does GC work

GC identifies objects that are no longer reachable from "GC Roots" and reclaims their memory. Modern collectors use generational hypothesis: most objects die young. Garbage Collection (GC) in Java is the process by which the JVM automatically manages memory by reclaiming memory occupied by objects that are no longer in use by the application. This eliminates the need for manual memory management (like `malloc/free` in C), significantly reducing memory leaks and segmentation faults.

The Fundamental Logic

The GC identifies "garbage" using a **Reachability Analysis** (or "Mark-and-Sweep" principle):

1. **GC Roots:** The JVM identifies "roots" (active local variables, static variables, active threads).
2. **Marking:** It traverses the object graph starting from these roots. Any object reachable from these roots is considered "alive."
3. **Sweeping:** Any object not reachable from the roots is considered "garbage" and its memory is reclaimed.

The Generations (Heap Structure)

Java divides the Heap into "generations" based on the observation that **most objects die young**. This allows the GC to focus its efforts where the most memory is reclaimed.

- **Young Generation (Eden & Survivor Spaces):** Where new objects are created. This area is small, and the GC runs here frequently (**Minor GC**). It is very fast.
- **Old Generation (Tenured):** Objects that survive multiple garbage collection cycles in the Young Generation are moved here. This area is larger, and GC runs here less frequently (**Major/Full GC**).

The GC Lifecycle

1. **Allocation:** New objects are created in the **Eden Space**.
2. **Minor GC:** When Eden is full, the GC moves surviving objects to a **Survivor Space**.
3. **Promotion:** Objects that survive enough minor collections are "promoted" to the **Old Generation**.
4. **Major/Full GC:** When the Old Generation fills up, a "Major GC" is triggered to reclaim memory from the entire heap. This is typically a "Stop-the-World" event, meaning the application pauses execution until cleanup is complete.

Types of Garbage Collectors

The JVM offers different GC algorithms to balance throughput versus latency:

- **Serial GC:** Single-threaded. Best for small applications or embedded systems.
- **Parallel GC:** Uses multiple threads for minor GC. Best for throughput-oriented batch processing.

- **G1 (Garbage First) GC:** The modern default. It divides the heap into regions and intelligently collects the regions with the most garbage first, aiming to meet user-defined latency goals.
- **ZGC / Shenandoah:** Ultra-low-latency collectors designed to keep pause times consistently under 1ms, even for huge heaps.

Performance Impact

The main goal of tuning GC is to **minimize the duration and frequency of "Stop-the-World" pauses**.

- If your application pauses frequently, it may indicate your heap is too small, causing constant GC.
- If your pauses are extremely long, your heap might be too large, or you might be generating too many short-lived objects.

Pro-Tip: You can monitor GC behavior in real-time by adding `-Xlog:gc*` to your Java startup arguments. This will print detailed logs about every collection event, which is essential when diagnosing performance bottlenecks in production.

34. Young/Old/Perm generation

- **Young Gen:** Newly created objects (frequent collections).
- **Old Gen:** Objects that survived many cycles.
- **Perm/Metaspace:** Stores class metadata.

To understand memory management in Java, you must look at how the JVM organizes the **Heap** and **Metaspace**. Java divides memory into specific "generations" based on the **Weak Generational Hypothesis**, which states that most objects "die young" (become eligible for garbage collection shortly after creation).

1. Young Generation

This is where all new objects are born. It is designed for high-frequency, fast cleanup.

- **Structure:** Divided into three sub-spaces:
 - **Eden Space:** The initial allocation area. When this fills up, a "Minor GC" occurs.
 - **Survivor Spaces (S0 & S1):** Objects that survive a Minor GC in Eden are moved here. They toggle between S0 and S1 with each cycle.
- **Behavior:** Because most objects become unreachable quickly, garbage collection here is extremely fast and happens frequently.

2. Old (Tenured) Generation

When an object survives a specific number of GC cycles in the Young Generation (the "tenure threshold"), it is promoted to the Old Generation.

- **Purpose:** This area holds long-lived objects (e.g., cached data, application-wide configuration objects, or thread pools).
- **Behavior:** GC runs here much less frequently than in the Young Gen. When it does occur, it is often a **Major GC** or **Full GC**, which is more time-consuming because there is more memory to scan.

3. Permanent Generation / Metaspace

The "PermGen" is a legacy concept that has been replaced. It is important to know the difference:

- **Permanent Generation (Java 7 and earlier):** Stored class metadata, static variables, and interned strings. It had a fixed maximum size, often leading to the dreaded `java.lang.OutOfMemoryError: PermGen space`.
- **Metaspace (Java 8 and later):** The official replacement for PermGen. The key difference is that **Metaspace is stored in native memory** (outside the JVM heap). It automatically resizes itself to prevent the fixed-size limitations of the old PermGen.

Why this division matters

By segregating objects by their "age," the JVM can employ different strategies:

1. **Young Gen GC** can be very aggressive and stop the world for only tiny fractions of a second because it only needs to look at a small portion of memory.
2. **Old Gen GC** can be more thorough, as these objects are expected to stay around for a long time.
3. **Metaspace** (Native) is separate, so loading thousands of classes is less likely to crash your application heap.

Pro-Tip: If you see high frequency of Full GCs, it usually means your **Old Generation** is filling up, often because the **Young Generation** is too small to handle the inflow of objects, causing them to be promoted (tenured) prematurely.

35. Different types of GC

Includes Serial, Parallel, CMS (Concurrent Mark Sweep), and G1 (Garbage First). Each balances throughput vs. pause time differently.

Garbage Collection (GC) is not a "one-size-fits-all" solution. The Java Virtual Machine (JVM) provides several different GC algorithms, each tuned for a different balance between **throughput** (total work done by the application) and **latency** (how long the application pauses).

1. Serial GC

The simplest collector. It uses a **single thread** for all garbage collection tasks.

- **Mechanism:** When it runs, it triggers a "Stop-the-World" (STW) event, freezing all application threads.
- **Use Case:** Best for small, single-threaded applications (e.g., CLI tools) or environments with limited memory (client-side Java).
- **Flag:** `-XX:+UseSerialGC`

2. Parallel GC (Throughput Collector)

Uses **multiple threads** to perform minor and major garbage collection.

- **Mechanism:** Designed to maximize throughput. It freezes application threads during collection, but since it uses multiple CPU cores, the collection phase is much faster than Serial GC.
- **Use Case:** Best for back-end applications where batch processing is more important than short, millisecond-level pauses (e.g., data processing, reporting).
- **Flag:** `-XX:+UseParallelGC`

3. G1 GC (Garbage First)

The **default collector** for Java 9 and later. It is designed to be the "all-rounder."

- **Mechanism:** It divides the heap into many small, equal-sized regions. It tracks the "liveness" of these regions and prioritizes collecting the ones that are "mostly garbage" first. This allows it to meet **pause-time goals** defined by the developer.
- **Use Case:** Suitable for most modern applications, especially those with large heaps (multi-gigabyte) that require a balance between throughput and responsiveness.
- **Flag:** `-XX:+UseG1GC`

4. ZGC (Z Garbage Collector) and Shenandoah

These are the "ultra-low-latency" collectors.

- **Mechanism:** They perform almost all of their heavy lifting (marking and compacting) **concurrently** with the running application threads.
- **Use Case:** Designed for massive heaps (terabytes) where even a 100ms pause would be unacceptable (e.g., real-time trading systems, high-frequency web services).
- **Latency:** They aim for pause times consistently under **1 millisecond**.
- **Flag:** `-XX:+UseZGC` or `-XX:+UseShenandoahGC`

How to choose the right one?

1. **Start with the Default:** For most Java 11/17/21+ applications, the default G1 GC is excellent.

2. **Monitor:** Use tools like `jstat`, `VisualVM`, or GC logs to measure your current pause times.
3. **Tweak:**
 - If you need **better throughput** and don't mind longer pauses, try `ParallelGC`.
 - If you are hitting **latency issues** (users experiencing lag) and have a large heap, move to `ZGC`.

Pro-Tip: You can see which collector your JVM is using by default by running:
`java -XX:+PrintCommandLineFlags -version`. Look for the flag that specifies the GC.

36. What is Future

`Future` is an interface representing the result of an asynchronous computation. It allows you to query if the task is complete, wait for it, and retrieve the result.

In Java, a `Future` (part of the `java.util.concurrent` package) represents the **result of an asynchronous computation**. Think of it as a "placeholder" or a "receipt" for a value that hasn't been calculated yet but will be available at some point in the future.

How it Works

When you submit a task (like a time-consuming calculation or a network request) to an `ExecutorService`, the service returns a `Future` object immediately. You can continue doing other work while the task runs in the background. Later, you can check on the status of the task or wait for it to complete.

Core Methods

- `get()`: Waits if necessary for the computation to complete, and then retrieves its result. **Warning:** This is a blocking call—your current thread will stop and wait until the result is ready.
- `get(long timeout, TimeUnit unit)`: Waits for at most the specified time for the computation to complete, then retrieves its result. This is much safer than the standard `get()` to avoid infinite hangs.
- `isDone()`: Returns `true` if the task completed (normally, via exception, or cancellation).
- `cancel(boolean mayInterruptIfRunning)`: Attempts to stop the task.

Limitations of Future

While `Future` is useful, it has significant limitations:

1. **Blocking:** The `get()` method blocks the thread, which defeats the purpose of non-blocking asynchronous programming.
2. **No Chaining:** You cannot easily pipe the result of one `Future` into another without nesting them, which leads to "callback hell."
3. **No Exception Handling:** There is no built-in way to handle exceptions functionally (like `try-catch` blocks for streams).

The Evolution: `CompletableFuture` (Java 8+)

Because of these limitations, Java 8 introduced `CompletableFuture`. It allows you to:

- **Chain tasks:** Use `.thenApply()` or `.thenCompose()` to run tasks sequentially.
- **Combine tasks:** Use `.thenCombine()` to run two tasks in parallel and merge their results when both are done.
- **Non-blocking:** Use callback methods (like `.thenAccept()`) that trigger automatically when the task finishes, without blocking your main thread.

Key Takeaway: If you are using Java 8 or higher, you should almost always prefer `CompletableFuture` over the basic `Future` interface. It provides a much more powerful and flexible API for building reactive, non-blocking applications.

37. What is CompletableFuture

`CompletableFuture` is a powerful implementation of `Future` that supports functional programming, callback-based triggers, and chaining (e.g., `thenApply`, `thenCompose`), avoiding manual polling.

`CompletableFuture` (introduced in Java 8) is a powerful evolution of the standard `Future`. While a basic `Future` is just a "placeholder" for a result that forces you to block and wait for it, a `CompletableFuture` is a **complete framework** for asynchronous, non-blocking programming.

It allows you to define a pipeline of tasks that run in the background and react to each other without ever needing to block the main thread.

Why `CompletableFuture` is a Game-Changer

1. **Non-Blocking Callbacks:** You can define what happens *after* a task completes using methods like `.thenApply()`, `.thenAccept()`, and `.thenRun()`.
2. **Task Chaining:** You can link asynchronous tasks together, where the output of one task becomes the input of the next.
3. **Combination:** You can run multiple asynchronous tasks in parallel and wait for them to finish, or trigger a new action only when *all* (or *any*) of them complete.
4. **Exception Handling:** It provides functional ways to handle errors using `.exceptionally()` or `.handle()`.

When to use which?

- **`supplyAsync`:** When you have a task that returns a result.
- **`runAsync`:** When you have a task that just performs an action (returns `void`).
- **`thenApply` vs `thenCompose`:** Use `thenApply` if your function returns a simple value. Use `thenCompose` if your function returns another `CompletableFuture` (to avoid returning a "Future of a Future").

Best Practices

- **Use a Custom Executor:** By default, `CompletableFuture` uses the `ForkJoinPool.commonPool()`. In production systems, you should provide a **custom `ExecutorService`** to avoid thread starvation if your tasks involve I/O or blocking operations.
- **Handle Exceptions:** Always use `exceptionally()` or `handle()` to ensure your application doesn't fail silently or hang forever if a background task crashes.
- **Avoid `get()`:** The whole point of `CompletableFuture` is to be non-blocking. Only use `get()` or `join()` as a last resort at the very top level of your application (like in the `main` method or a controller layer).

Crucial Concept: `CompletableFuture` is fundamentally about **completing** the chain of events. Because it implements the `CompletionStage` interface, it allows for a highly readable "fluent" API that looks more like a standard stream of operations than traditional multi-threaded code.

38. Future vs CompletableFuture

`Future` is passive and blocking (must use `.get()`). `CompletableFuture` is active and non-blocking, allowing you to define completion logic or chain multiple tasks together seamlessly.

Follow-up: Which of these areas would you like to deep-dive into first with a coding demo or a more detailed architectural explanation?

The primary difference between `Future` and `CompletableFuture` is the transition from **blocking, passive waiting** to **asynchronous, reactive programming**.

While both represent the result of an operation that hasn't finished yet, they provide fundamentally different tools for handling that result.

1. The `Future` Approach (The "Waiter")

The standard `Future` interface is **imperative**. You submit a task, and then you have to actively check if it's done.

- **Blocking**: When you call `future.get()`, your code stops and waits. If you do this on the main thread, the entire application freezes until the result arrives.
- **Disconnected**: You cannot define what should happen after the task completes without explicitly adding code to check for completion.

2. The `CompletableFuture` Approach (The "Orchestrator")

`CompletableFuture` is **declarative**. Instead of waiting, you describe the *entire workflow* of what should happen to the data once it becomes available.

- **Non-Blocking**: You define callbacks (like `thenApply`). The thread that submitted the task is free to do other work immediately.
- **Pipeline-oriented**: You can chain methods to transform data, catch errors, or merge results from multiple background tasks.

When to choose which?

- **Choose `Future`** only if you are working on a very simple application where you just need to fire off a single task and block until the result returns. It is lightweight but extremely limited.
- **Choose `CompletableFuture`** for modern Java development. It is the standard for building responsive, scalable, and non-blocking systems.

Key Difference in Philosophy: With `Future`, you are **asking** the background task for the result ("Are you done yet?"). With `CompletableFuture`, you are **telling** the background task what to do once it finishes ("When you are done, do X, then Y, then Z").